

Guard App - Automated Testing Research

Introduction

As part of this task, I researched automated testing approaches for React Native applications to understand how testing can be introduced into the Guard App. Automated testing is important because it helps ensure that the application works correctly without needing to manually test every feature after changes are made. It also helps identify bugs early and improves overall code reliability.

Tools that I explored

During my research, I explored a few different tools that are commonly used for automated testing in React Native.

Jest

Jest is a JavaScript testing framework that is widely used for unit and component testing. It is also the default testing framework recommended for React Native projects.

From my research, I found that Jest is useful because it is easy to set up and allows developers to test individual parts of the application, such as functions and components. It also supports mocking, which is helpful when testing features that depend on external data.

React Native Testing Library

React Native Testing Library is used alongside Jest to test user interface components. It focuses on testing how the user interacts with the app rather than how the code is written internally.

This makes it more practical for mobile applications because it allows testing of things like button presses, text inputs, and screen rendering.

Detox

I also explored Detox, which is an end-to-end testing tool. Detox allows full application testing, such as simulating a user logging in and navigating through different screens.

However, based on my research, Detox requires more setup and is more complex compared to Jest and React Native Testing Library. Because of this, it may be better suited for later stages of development when more comprehensive testing is required.

Maestro

Another tool I looked into was Maestro, which is also used for end-to-end testing. Maestro stood out because it is easier to set up compared to other E2E tools and supports cross-platform testing.

From what I found, Maestro is a strong option for testing complete user flows in a simple way. If more time was available, I would like to further explore Maestro and potentially implement it to test full application behaviour, such as navigation and multi-step user interactions.

Comparison

To better understand which tool would be most suitable, I compared the options based on factors such as ease of setup, learning curve, and type of testing supported.

Tool	Type of Testing	Ease of Setup	Key Strengths	Limitations
Jest	Unit / Component	Easy	Fast, built-in support, widely used	Limited for full app testing
React Native Testing Library	Component / UI	Easy	Tests user interactions, simple syntax	Requires Jest
Detox	End-to-End	Moderate-Hard	Full app flow testing, realistic scenarios	Complex setup and maintenance
Maestro	End-to-End	Moderate	Simple E2E testing, cross-platform, readable flows	Less widely used, still evolving

From this comparison, it is clear that different tools are suited for different purposes. Jest and React Native Testing Library are more suitable for testing individual components and interactions, while Detox and Maestro are more focused on testing complete user flows.

Decision and Justification

After comparing the tools, I decided to focus on Jest and React Native Testing Library for this stage of the project.

This decision was based on the following reasons:

- These tools are easier to set up and integrate into the existing React Native project
- They are sufficient for testing core functionality such as UI rendering and user interactions
- They are widely used and well-documented, making them easier to learn and apply

- They allow testing to be introduced incrementally without requiring major changes to the project

While Detox and Maestro provide more advanced testing capabilities, they require additional setup and time to implement effectively.

However, from my research, Maestro appears to be a strong option for end-to-end testing due to its simpler configuration and ability to simulate real user flows. If more time is allocated in the future, I would like to explore Maestro further and potentially use it to test complete workflows such as login and navigation.

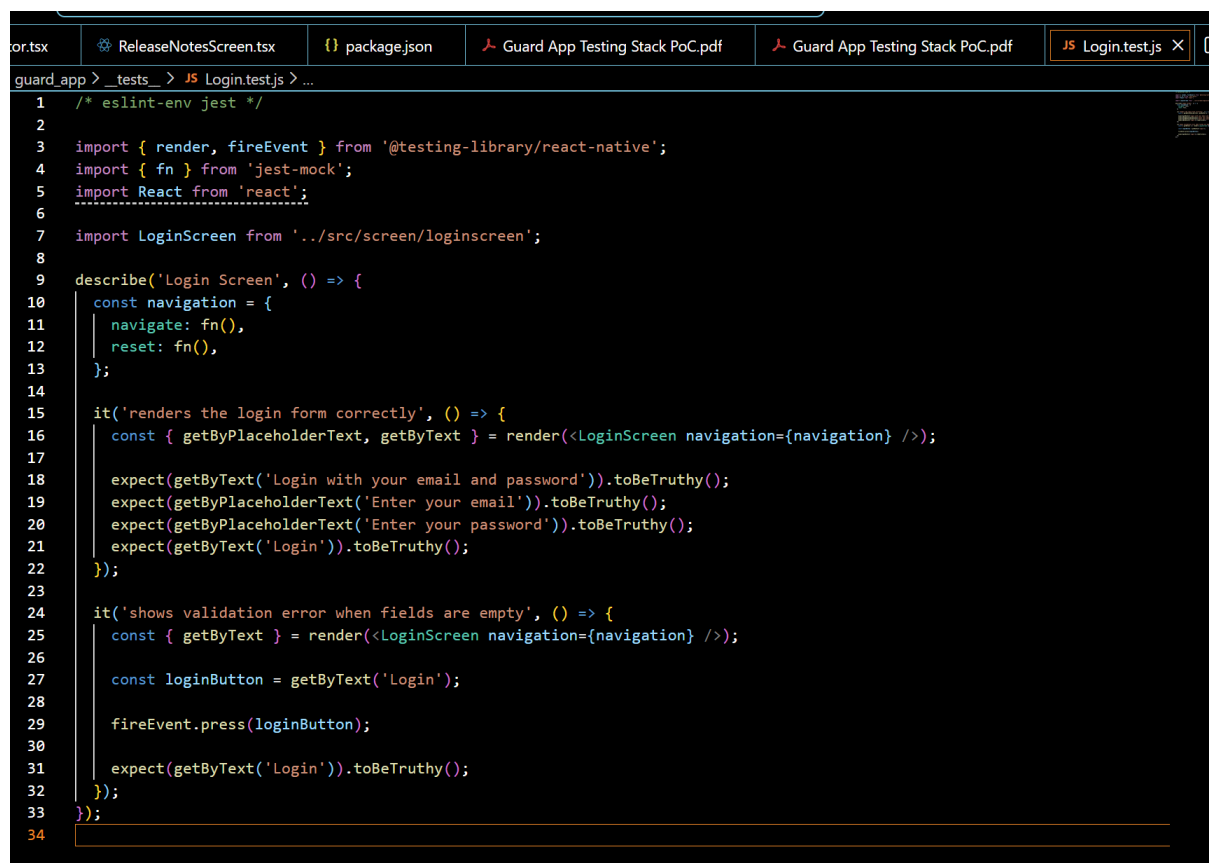
Key Observations

Through this research, I learned that:

- Automated testing is important for maintaining application quality
- Different testing types (unit, component, end-to-end) serve different purposes
- Testing user interactions is more effective than testing internal implementation
- Simpler tools should be used first before moving to more complex solutions

Practical Exploration

1: Test File



```

1  /* eslint-env jest */
2
3  import { render, fireEvent } from '@testing-library/react-native';
4  import { fn } from 'jest-mock';
5  import React from 'react';
6
7  import LoginScreen from '../src/screen/loginscreen';
8
9  describe('Login Screen', () => {
10   const navigation = {
11     navigate: fn(),
12     reset: fn(),
13   };
14
15   it('renders the login form correctly', () => {
16     const { getByPlaceholderText, getByText } = render(<LoginScreen navigation={navigation} />);
17
18     expect(getByText('Login with your email and password')).toBeTruthy();
19     expect(getByPlaceholderText('Enter your email')).toBeTruthy();
20     expect(getByPlaceholderText('Enter your password')).toBeTruthy();
21     expect(getByText('Login')).toBeTruthy();
22   });
23
24   it('shows validation error when fields are empty', () => {
25     const { getByText } = render(<LoginScreen navigation={navigation} />);
26
27     const loginButton = getByText('Login');
28
29     fireEvent.press(loginButton);
30
31     expect(getByText('Login')).toBeTruthy();
32   });
33 });
34

```

This shows the structure of the test cases, including how components are rendered and how user interactions are tested.

2: Running Tests and Results

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  TOOLS

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

    at node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11801:19
    at runWithFiberInDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:1574:13)
    at warnIfUpdatesNotWrappedWithActDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11800:9)
    at scheduleUpdateOnFiber (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:10326:9)
    at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)

    at node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11801:19
    at runWithFiberInDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:1574:13)
    at warnIfUpdatesNotWrappedWithActDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11800:9)
    at scheduleUpdateOnFiber (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:10326:9)
    at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)
enderer.development.js:11800:9)
    at scheduleUpdateOnFiber (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:10326:9)
    at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)
    at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)
elopment.js:14179:14)
    at Icon.Object.<anonymous>.process.env.NODE_ENV.Component.setState (node_modules/react/cjs/react.development.js:632:20)
    at Icon.setState (node_modules/@expo/vector-icons/src/createIconSet.tsx:181:31)
    at asyncGeneratorStep (node_modules/@babel/runtime/helpers/asyncToGenerator.js:3:17)
    at Icon.setState (node_modules/@expo/vector-icons/src/createIconSet.tsx:181:31)
    at asyncGeneratorStep (node_modules/@babel/runtime/helpers/asyncToGenerator.js:3:17)
    at _next (node_modules/@babel/runtime/helpers/asyncToGenerator.js:17:9)

PASS  __tests__/_Login.test.js (7.479 s)
    at asyncGeneratorStep (node_modules/@babel/runtime/helpers/asyncToGenerator.js:3:17)
    at _next (node_modules/@babel/runtime/helpers/asyncToGenerator.js:17:9)

PASS  __tests__/_Login.test.js (7.479 s)
  Login Screen
    at _next (node_modules/@babel/runtime/helpers/asyncToGenerator.js:17:9)

PASS  __tests__/_Login.test.js (7.479 s)
  Login Screen
    ✓ renders the login form correctly (2668 ms)

PASS  __tests__/_Login.test.js (7.479 s)
  Login Screen
    ✓ renders the login form correctly (2668 ms)
    ✓ renders the login form correctly (2668 ms)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 8.263 s
Ran all test suites.
PS C:\SecureShift1\SecureShift\guard-app> npm test
```

```
PS C:\SecureShift1\SecureShift\guard_app> npm test

> secureshift-guardapp@1.0.0 test
> jest

[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: `npm i baseline-browser-mapping@latest -D`
An update to Icon inside a test was not wrapped in act(...).

When testing, code that causes React state updates should be wrapped into act(...):

act(() => {
  /* fire events that update state */
});
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

at node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11801:19
at runWithFiberInDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:1574:13)
at warnIfUpdatesNotWrappedWithActDEV (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:11800:9)
at scheduleUpdateOnFiber (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:10326:9)
at Object.enqueueSetState (node_modules/react-test-renderer/cjs/react-test-renderer.development.js:14179:14)
at Icon.Object.<anonymous>.process.env.NODE_ENV.Component.setState (node_modules/react/cjs/react.development.js:632:20)
at Icon.setState (node_modules/@expo/vector-icons/src/createIconSet.tsx:181:31)
at asyncGeneratorStep (node_modules/@babel/runtime/helpers/asyncToGenerator.js:3:17)
at _next (node_modules/@babel/runtime/helpers/asyncToGenerator.js:17:9)

PASS _tests_/Login.test.js
  Login Screen
    ✓ renders the login form correctly (354 ms)
    ✓ shows validation error when fields are empty (280 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 2.433 s, estimated 8 s
Ran all test suites.
PS C:\SecureShift1\SecureShift\guard_app>
```

These screenshots show the execution of automated tests using Jest including the test results confirming that all test cases are passing successfully.

3: Jest Setup

```
package.json  Guard App Testing Stack PoC.pdf  Guard App Testing Stack PoC.pdf  JS Login.test.js  eslintrc.js  JS jest.setup.js  [ ]
guard_app > JS jest.setup.js > ...
1 import mockAsyncStorage from '@react-native-async-storage/async-storage/jest/async-storage-mock';
2
3 jest.mock('@react-native-async-storage/async-storage', () => mockAsyncStorage);
4
```

This shows how dependencies such as AsyncStorage are mocked to allow tests to run correctly.

Conclusion

Overall, automated testing is a valuable practice that can improve the reliability and maintainability of the Guard App. Based on the research, Jest and React Native Testing Library are suitable tools for introducing automated testing due to their simplicity and effectiveness.

More advanced tools such as Maestro provide strong support for end-to-end testing and could be explored further if more time is allocated in the future.

References

Callstack. (n.d.). *React Native Testing Library documentation*. <https://callstack.github.io/react-native-testing-library/>

Jest. (n.d.). *Jest documentation*. <https://jestjs.io/docs/getting-started>

Detox. (n.d.). *Detox testing framework*. <https://wix.github.io/Detox/>

Maestro. (n.d.). *Maestro mobile testing framework*. <https://maestro.mobile.dev/>